# Stringhe in C

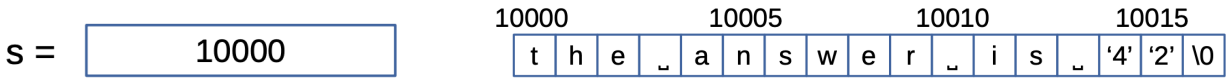
| Description | Come vengono memorizzate stringhe in C, scanf() |
|---|---|

## Stringhe in C

Una stringa è un array di caratteri, in questo caso speciale non si memorizza la lunghezza ma si usa la convenzione che la stringa termina quando si incontra il byte 0.

Una stringa ha sempre una lunghezza di s + 1 in quanto si tiene conto del `\0`

```
char *s = “the answer is 42”;
```




`\0` fa capire al *`printf()`* o *`puts()`* che la stringa finisce qui

```
scanf(“<StringaDiFormato>”,<Variabile>)

StringaDiFormato rappresenta lo specificatore di formato ed è preceduto,
                 come nel printf, dal simbolo %.

Variabile rappresenta il nome della variabile all’interno della quale è
          memorizzato il dato digitato da tastiera.
```

```
int e = scanf("%d",&n);
                    ^ Inserisco nell'indirizzo di memoria di n il valore
                      digitato dall'utente
```

## `Scanf` con stringhe

```
scanf("%d", "%11s", &numero, &stringaDi_11Caratteri)
```

📌 **La sequenza di Escape:**

La sequenze di escape sono dei caratteri che non vengono visualizzati, ma contribuiscono a formattare il testo. Tutte le sequenze di escape iniziano con il carattere di backslash (\) e sono interpretate come un singolo carattere.

Esempio:
**\n** nuova riga (new line), cioè va a capo
**\t** fa uno spazio con tab

## `strcmp()`

Funzione dedicata al confronto tra due stringhe

```
strcmp(first_str, second_str );
```

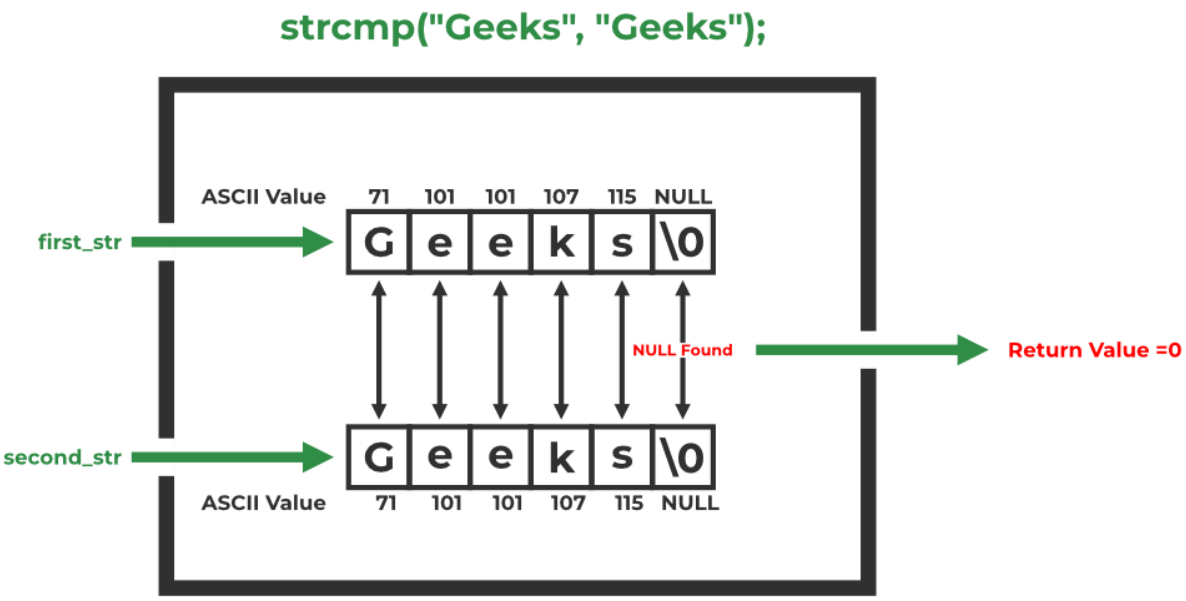
per utilizzarla bisogna includere la seguente libreria:

```
#include <string.h>
```

📌 ***first_str*** La prima stringa viene presa come puntatore al carattere costante ***second_str*** La seconda stringa viene presa come puntatore a un carattere costante.

## Valori di ritorno di `strcmp()`

- `0` - Un valore uguale a zero quando entrambe le stringhe risultano identiche. Cioè, tutti i caratteri in entrambe le stringhe sono gli stessi.
- `>0` - Viene restituito un valore maggiore di zero quando il primo carattere non corrispondente in first_str ha un valore ASCII maggiore del carattere corrispondente in second_str
- `<0` - Viene restituito un valore inferiore a zero quando il primo carattere non corrispondente in first_str ha un valore ASCII inferiore rispetto al carattere corrispondente in second_str.




ESEMPIO:

```
#include <stdio.h>
#include <string.h>

int main()
{
```

```
  // declaring two same string
  char* first_str = "Geeks";
  char* second_str = "Geeks";

  // printing the strings
  printf("First String: %s\n", first_str);
  printf("Second String: %s\n", second_str);

  // printing the return value of the strcmp()
  printf("Return value of strcmp(): %d",
    strcmp(first_str, second_str));

  return 0;
}
```

OUTPUT:

```
First String: Geeks
Second String: Geeks
Return value of strcmp(): 0
```